

JavaScript Başlangıç Notları

Değişkenler, Stringler, Sayılar, Diziler ve Nesneler

Ayşenur

3 Ağustos 2026

Bu belge, JavaScriptBaslangic GitHub reposundaki
3_js_kodlama klasöründe yazılan pratik kodların ders notu haline getirilmiş halidir.

İçindekiler

Klasör Yapısı	2
1 script.js — Değişkenler, Veri Tipleri, String Temelleri	3
2 script2.js — String Uygulamaları	5
3 script3.js — Sayılar ve Diziler	6
4 script4.js — Dizi Metotları	7
5 script5.js — Dizi Uygulaması + Nesneler (Objects)	8
6 script6.js — Nesne Uygulaması: Sipariş Sistemi	9
7 Genel Özet — Öğrenilen Konular	10
8 Tekrar Ederken Dikkat Edilecek Noktalar	10

Klasör Yapısı

```
3_js_kodlama/  
|-- index.html -> script6.js'i sayfaya bağlayan basit HTML dosyası  
|-- script.js -> Değişkenler, veri tipleri, string temelleri  
|-- script2.js -> String uygulamaları (URL/slug örneği)  
|-- script3.js -> Sayılar (Number, Math) ve diziler  
|-- script4.js -> Dizi metodları  
|-- script5.js -> Dizi uygulaması + nesneler (objects)  
`-- script6.js -> İç içe nesne/dizi uygulaması (sipariş sistemi)
```

Konsol çıktılarını görmek için `index.html` dosyasını tarayıcıda açıp geliştirici araçları (F12) → Console sekmesine bakman yeterli.

1 script.js — Değişkenler, Veri Tipleri, String Temelleri

Değişken Tanımlama

var anahtar kelimesiyle değişken tanımlandı (let de aynı işi görür, aralarındaki fark ileride kapsam/scope konusunda önemli olacak).

```
var a = 5000;
a = 6000; // var ile tanımlanan degiskenin degeri sonradan degistirilebilir
```

Değişken isimlendirme kuralları: İsimler rakamla başlayamaz, JS'in ayrılmış komut isimleri (var, function vb.) değişken adı olarak kullanılamaz.

Veri Tipleri ve typeof

```
var urunAdi = 'Iphone 16'; // string
var urunFiyat2 = 7000; // number
console.log(typeof urunAdi); // "string"
```

İki string + ile birleştirildiğinde matematiksel toplama değil, **concatenation (birleştirme)** yapılır:

```
var sayi1 = '10';
var sayi2 = '20';
console.log(sayi1 + sayi2); // "1020" olur, 30 değil!
```

undefined: Değer atanmamış bir değişkenin hem değeri hem tipi undefined döner.

Uygulama — Öğrenci Not Ortalaması

İki öğrencinin doğum yılından yaşını hesaplama (2026 - doğumYılı) ve üç sınav notunun ortalamasını alıp 50 barajına göre başarı durumunu (true/false) belirleme. Bu, karşılaştırma operatörlerinin (>=) boolean sonuç ürettiğini gösteren güzel bir örnek.

String'e Index ile Erişim ve Template String

```
var ad = 'Aysenur';
console.log(ad[0]); // "A"

var mesaj = `Benim adim ${ad}`; // template string (backtick)
```

String Metotları

Metot	Ne yapar
toUpperCase()	Tüm harfleri büyütür
length	Karakter sayısını verir
slice(start, end)	Belirtilen aralığı keser; negatif index sağdan sayar
substring(start, end)	slice'a benzer, negatif index desteklemez
replace(eski, yeni)	İlk eşleşmeyi değiştirir
trim()	Baştaki/sondaki boşlukları siler
indexOf(metin)	Metnin bulunduğu index'i verir, yoksa -1
split(ayraç)	Stringi diziye böler

Dosyada `kursAdi.split(" ")[-1]` ile son elemanı almaya çalışılmış. JavaScript'te Python'daki gibi **negatif index desteği yoktur**, bu yüzden `[-1]` çalışmaz ve `undefined` döner. Son elemanı almak için `split(" ").at(-1)` ya da `split(" ")[dizi.length - 1]` kullanılmalı.

2 script2.js — String Uygulamaları

Gerçek bir senaryo: bir URL ve kurs adından “slug” (URL’de kullanılabilir kısa isim) üretme.

```
let url = "https://www.sadikturan.com";
let kursAdi = "Komple Web Gelistirme Kursu";

var kacKelime = kursAdi.split(" "); // kelimelere ayirma
var urlBulma = url.split(":"); // protokolu ayirma ("https")
var kucuk = kursAdi.toLowerCase(); // kucuk harfe cevirme
var yeni = kucuk.replaceAll(" ", "-"); // bosluklari tire ile degistirme

var mesaj = `${url}/${yeni}`;
console.log(mesaj); // https://www.sadikturan.com/komple-web-gelistirme-kursu
```

Bu dosyada öğrenilen ana fikir: string metotları zincirleme kullanılarak gerçek dünyada işe yarayan bir dönüşüm (slug oluşturma) yapılabilir.

3 script3.js — Sayılar ve Diziler

Tip Dönüştürme ve Biçimlendirme

```
sonuc = Number(sonuc); // string -> number
sonuc = parseFloat("20.2"); // ondalıklı stringi sayıya çevirir

let sayi = 10.58946;
sayi.toFixed(6); // toplam 6 basamak gösterir
sayi.toFixed(5); // virgülden sonra 5 basamak gösterir
```

Math Nesnesi

Fonksiyon	Ne yapar
Math.round(x)	En yakın tam sayıya yuvarlar
Math.ceil(x)	Yukarı yuvarlar
Math.floor(x)	Aşağı yuvarlar
Math.sqrt(x)	Karekök alır
Math.pow(x, y)	Üs alma (x^y)
Math.min(...)	En küçük değeri bulur
Math.random()	0 ile 1 arasında rastgele sayı üretir

$\text{Math.random()} * 10 \rightarrow 0-10$ arası, $\text{Math.random()} * 100 + 1 \rightarrow 1-101$ arası sayı üretmek için kullanılan genel kalıp.

Diziler (Array) ve İç İç Diziler

```
let urunler = ["Iphone 15", "Iphone 16", "Iphone 17"];
let fiyatlar = [50000, 60000, 70000];

let urun11 = ["Iphone 15", 50000, ["gold", "silver", "black"]];
console.log(urun11[2][2]); // "black" -> iç dizinin 3. elemanı
```

4 script4.js — Dizi Metotları

Bir öğrenci listesi üzerinden dizilerde ekleme/çıkarma/arama işlemleri denenmiş:

Metot	Ne yapar
length	Eleman sayısı
toString()	Diziyi virgülle ayrılmış stringe çevirir
join(' ')	Diziyi belirtilen ayraçla stringe çevirir
pop()	Son elemanı siler ve döndürür
shift()	İlk elemanı siler ve döndürür
push(x)	Sona eleman ekler , dizinin yeni uzunluğunu döndürür
unshift(x)	Başta eleman ekler , dizinin yeni uzunluğunu döndürür
indexOf(x)	Elemanın index'ini bulur
includes(x)	Eleman var mı diye true/false döner
splice(b, s, ...y)	Belirtilen konumdan eleman siler/ekler

`push()` ve `unshift()` eklenen elemanı değil, **dizinin yeni uzunluğunu (length)** döndürür. Kodda `sonuc = ogrenciler.push("Sena")` satırından sonra `console.log(sonuc)` bir sayı (dizinin yeni uzunluğu) basar, "Sena" değil — bu JS'e yeni başlayanların en çok şaşırdığı noktalardan biri.

5 script5.js — Dizi Uygulaması + Nesneler (Objects)

İlk kısımda meyve dizisi üzerinde `push/pop` pratiği yapılmış.

Nesneler (Object) — Key/Value Yapısı

```
let kullanıcı1 = {  
  "ad": "Sadık",  
  "soyad": "Turan",  
  "yas": 40,  
  "adres": {  
    "sehir": "kocaeli",  
    "ilce": "izmit"  
  },  
  "hobiler": ["sinema", "spor"]  
};
```

Nesne Değerlerine Erişim — İki Yöntem

```
kullanıcı1["ad"]; // köşeli parantez (bracket) notasyonu  
kullanıcı1.adres.ilce; // nokta (dot) notasyonu, iç içe nesnelerde de çalışır
```

"hobiler " anahtarının sonunda fazladan bir boşluk var (yazım hatası). Bu yüzden `kullanıcı1.hobiler` çalışmaz, sadece `kullanıcı1["hobiler "]` (boşlukla birlikte) çalışır. Nesne anahtarlarında boşluk/özel karakter olursa dot notation kullanılamaz, bracket notation zorunlu hale gelir — bu satır bunun canlı bir örneği. Ayrıca `kullanıcılar = [kullanıcı1, kullanıcı2];` satırında değişken `var/let` olmadan tanımlanmış. Bu, JS'te "implicit global" (örtük global değişken) oluşturur ve genelde önerilmez — ileride "use strict" modunda bu satır hataya bile sebep olabilir.

6 script6.js — Nesne Uygulaması: Sipariş Sistemi

Gerçekçi bir e-ticaret sipariş verisi üzerinden iç içe (nested) nesne ve dizi kullanımı pekiştirilmiş.

```
let siparis1 = {
  siparis_id: 101,
  satin_alinanlar: [ // dizi icinde nesneler (birden fazla urun)
    { urun_id: 5, urun_basligi: "Iphone 16 Pro", urun_fiyat: 75000 },
    { urun_id: 6, urun_basligi: "Iphone 16 Pro Max", urun_fiyat: 85000 }
  ],
  musterisi: { musterisi_id: 12 }
};
```

siparis2 içinde ise satin_alinanlar bir dizi değil, tek bir nesne — yani aynı alan farklı sipariş kayıtlarında farklı veri yapısına (dizi vs. tekil nesne) sahip olabiliyor, bu da erişim şeklini değiştiriyor:

```
// siparis1: dizi olduğu için index ile erişim + toplama
var siparis1_toplam = (siparis1.satin_alinanlar[0].urun_fiyat
  + siparis1.satin_alinanlar[1].urun_fiyat) * 1.2;

// siparis2: tekil nesne olduğu için doğrudan alan adıyla erişim
var siparis2_toplam = siparis2.satin_alinanlar.urun_fiyat * 1.2;
```

* 1.2 çarpanı %20 KDV eklemek için kullanılmış. Bu dosyanın öğrettiği en önemli fikir: gerçek dünya verisi (API'den gelen JSON gibi) genelde bu şekilde iç içe nesne ve dizilerin karışımından oluşur, bu yüzden veriye erişmeden önce yapısının (dizi mi, nesne mi) bilinmesi gerekir.

7 Genel Özet — Öğrenilen Konular

- Değişken tanımlama (`var`, `let`) ve isimlendirme kuralları
- Veri tipleri: `string`, `number`, `boolean`, `undefined` ve `typeof` operatörü
- String birleştirme (+) ile matematiksel toplama arasındaki fark
- Template literal (backtick) kullanımı
- String metotları: `toUpperCase`, `slice`, `substring`, `replace`, `trim`, `indexOf`, `split`
- Sayı dönüştürme (`Number`, `parseFloat`) ve biçimlendirme (`toFixed`, `toPrecision`)
- Math nesnesi fonksiyonları (`round`, `ceil`, `floor`, `sqrt`, `pow`, `min`, `random`)
- Diziler: tanımlama, `index` ile erişim, iç içe diziler
- Dizi metotları: `push`, `pop`, `shift`, `unshift`, `indexOf`, `includes`, `splice`, `join`
- Nesneler (objects): key-value yapısı, iç içe nesneler, dot/bracket notation
- Gerçekçi veri modelleme: dizi + nesne kombinasyonlarıyla (sipariş sistemi) çalışma

8 Tekrar Ederken Dikkat Edilecek Noktalar

- `split(" ")[-1]` JS'te çalışmaz — negatif index yoktur, `.at(-1)` kullanılmalı.
- `push()` / `unshift()` eklenen elemanı değil, dizinin **yeni uzunluğunu** döndürür.
- Nesne anahtarında boşluk/özel karakter varsa **sadece bracket notation** (`obj["anahtar adı"]`) çalışır, dot notation çalışmaz.
- Değişkeni `var/let` olmadan atamak örtük global değişken oluşturur, kaçınılmalı.
- Aynı alan (`satın_alinanlar` gibi) farklı kayıtlarda dizi ya da tekil nesne olabilir — erişimden önce veri yapısı kontrol edilmeli.